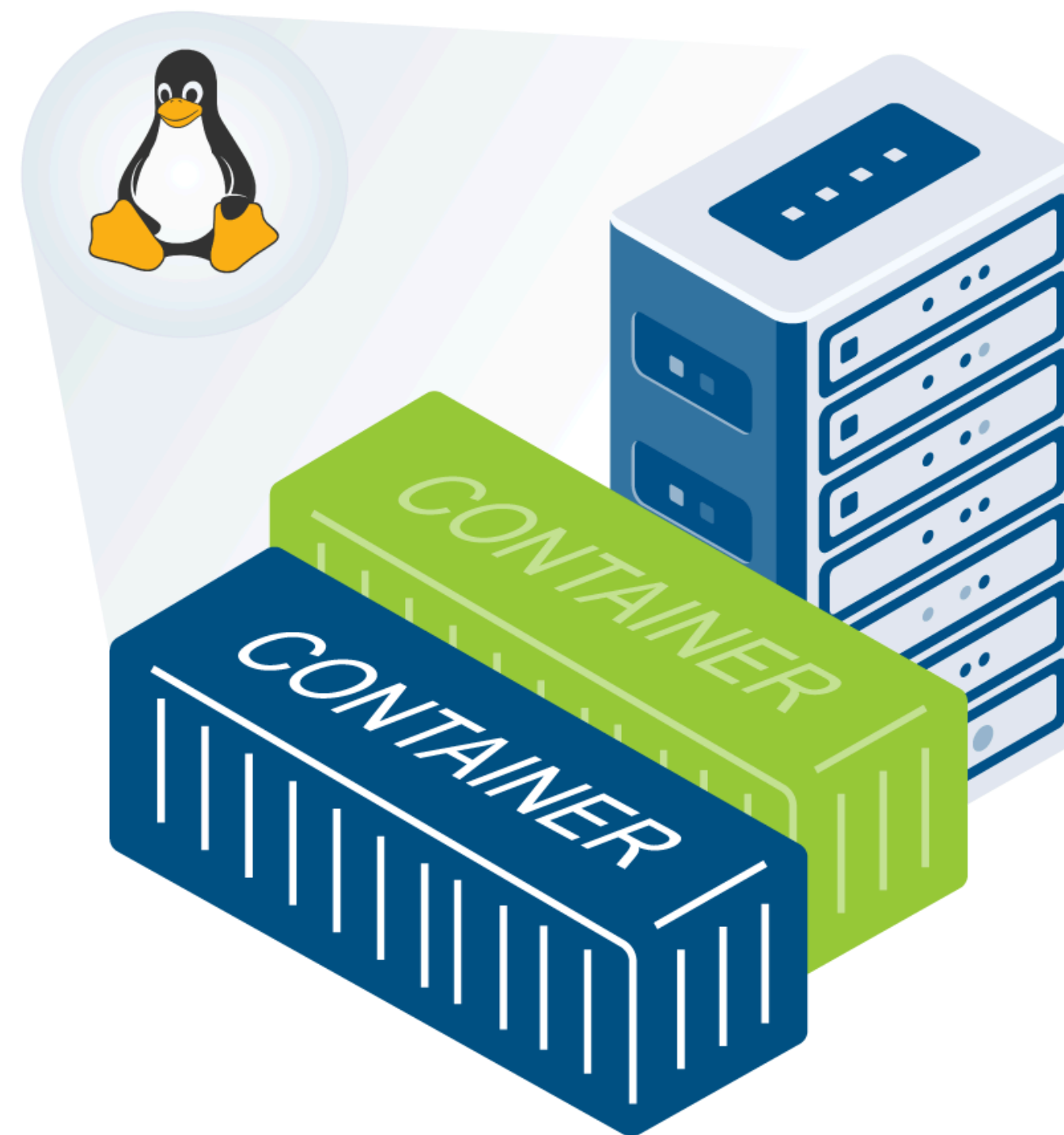


CONTAINERS FOR EMBEDDED LINUX; THE NEXT CHAPTER



Presented by

Drew Moseley

Torizon Professional Services, Staff Developer
Toradex

WITH YOU TODAY...



Drew Moseley

Torizon Professional Services, Staff Developer
Toradex

✉ drew.moseley@toradex.com

[in](#) /in/drewmoseley

✂ @drewmoseley

[drewmoseley](#)

[@drewmoseley@fosstodon.org](#)

WHY CONTAINERS *BEYOND* THE BASICS?



The basics are known. Embedded adds real constraints:



Storage

Flash is small



Network

Connectivity is unreliable



Power

Can fail at any time



Longevity

Devices live for years

IMAGE DESIGN

BEGINNER SETUP



- Single Dockerfile:
 - Full Debian image
 - Build tools included
 - Intermediate artifacts included
- Problems:
 - 498 MB image
 - Large attack surface
 - No separation of concerns
 - Runs as root

```
Dockerfile

FROM debian:trixie

RUN apt-get update && apt-get install -y \
    build-essential \
    nginx \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY sensor.c .
RUN gcc -o sensor sensor.c

COPY html/ /var/www/html/
COPY entrypoint.sh .
RUN chmod +x entrypoint.sh

EXPOSE 80

CMD ["/app/entrypoint.sh"]
```

Key idea: What you build with $\neq$ what you need to run.

STEP 0: RUNNING COMPARISON



- Toradex Verdin i.MX8M Mini + Dahlia Carrier Board, eMMC storage



↓ port 80 → Browser

Step	Change	Image Size
step0	Baseline: single-stage, debian:trixie, build tools included	498 MB

SMARTER BUILDS: MULTI-STAGE PATTERNS



- Multi-stage builds:
 - `builder` — toolchains, headers, debug tools
 - `runtime` — only what you need to run
- Benefits:
 - Smaller images
 - Fewer CVEs
 - Fewer packages on device
- Common patterns:
 - Strip binaries in builder
 - Separate debug vs production image

```
Dockerfile

# — Builder stage —
FROM debian:bookworm AS builder

RUN apt-get update && apt-get install -y \
    build-essential cmake libssl-dev

COPY src/ /src/
WORKDIR /src
RUN cmake -B build && \
    cmake --build build && \
    strip build/dashboard

# — Runtime stage —
FROM debian:bookworm-slim

COPY --from=builder \
    /src/build/dashboard \
    /usr/local/bin/

CMD ["/usr/local/bin/dashboard"]
```

Key idea: Use the builder as a workshop, the runtime as the shipping container.

Key idea #2: Container images as code.

STEP 1: RUNNING COMPARISON



- Same app, same board — only the Dockerfile changed



↓ port 80 → Browser

Step	Change	Image Size
step0	Baseline: single-stage, debian:trixie, build tools included	498 MB
step1	Multi-stage: debian:trixie-slim runtime, no build tools	112 MB

MICROSERVICES

- Monolithic problems:
 - Single failure domain
 - One crash takes everything down
 - Update one → rebuild all
 - No independent resource limits
- One process per container:
 - Own image, lifecycle, restart policy
 - Share data via **named volumes** or networking
 - Independent updates, sizing, security surface
- Common 3-service pattern: `sensor` + `api` + `frontend`
 - Own pipeline (React, Vue, ...)
 - Here: single static file — 2 services suffice

```
docker-compose.yml

services:
  sensor:
    image: dashboard-step2-sensor
    volumes:
      - dashboard-data:/data
    restart: unless-stopped

  nginx:
    image: dashboard-step2-nginx
    volumes:
      - dashboard-data:/data:ro
    ports:
      - "8080:80"
    restart: unless-stopped

volumes:
  dashboard-data:
```

Key idea: Containers as unit of deployment, not just packaging.

STEP 2: RUNNING COMPARISON



- Split into two containers: `sensor` writes JSON to a named volume; `nginx` reads it



Step	Change	Image Size
step0	Baseline: single-stage, debian:trixie, build tools included	498 MB
step1	Multi-stage: debian:trixie-slim runtime, no build tools	112 MB
step2	Microservices: sensor + nginx, named volume	101 + 112 MB = 213 MB naïve; ~112 MB on disk (shared base)

MINIMAL BASE IMAGES: DISTROLESS / SCRATCH



- Traditional bases bundle shell, package manager, libraries
- Minimal options:
 - **Distroless** (Google) — no shell, no package manager, just runtime libs
 - **Chiselled** (Canonical) — Ubuntu stripped to essentials
 - **scratch** — empty; static binaries only
- Embedded benefits:
 - Smaller flash footprint
 - Fewer packages to patch
 - Smaller CVE surface
- Distroless variants:
 - **distroless/base** — glibc (C/C++ programs)
 - **distroless/static** — statically compiled binaries only
 - **distroless/cc** — glibc + libstdc++ (C++ programs)

```
web/Dockerfile

# — Builder —
FROM golang:alpine AS builder

WORKDIR /build
COPY go.mod main.go ./
RUN CGO_ENABLED=0 GOOS=linux \
    go build -ldflags="-s -w" -o web .

# — Runtime: truly nothing —
FROM gcr.io/distroless/static

COPY --from=builder /build/web /web
COPY html/ /www/

CMD ["/web"]
```

Tradeoff: Great for production; need a separate path for debugging.

STEP 3: RUNNING COMPARISON



- Same architecture as step 2 — only the FROM lines changed



Step	Change	Image Size
step0	Baseline: single-stage, debian:trixie, build tools included	498 MB
step1	Multi-stage: debian:trixie-slim runtime, no build tools	112 MB
step2	Microservices: sensor + nginx, named volume	101 + 112 MB = 213 MB naïve; ~112 MB on disk (shared base)
step3	Distroless: sensor (35 MB) + Go web (7.6 MB)	~40 MB on disk

NON-ROOT CONTAINERS



- Root inside a container is still a risk:
 - Container escape → root on host
 - Accidental writes to system paths
- Run as an unprivileged user:
 - `USER` instruction in the Dockerfile
 - Distroless `:nonroot` — uid 65532, no extra steps
- App requirements:
 - No privileged ports (< 1024)
 - No writes to root-owned paths
 - Volumes writable by uid 65532

```
sensor/Dockerfile

# — Builder —
FROM debian:trixie-slim AS builder

RUN apt-get update && apt-get install -y \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

COPY sensor.c .
RUN gcc -o sensor sensor.c && strip sensor

# — Runtime: nonroot —
FROM gcr.io/distroless/base:nonroot

COPY --from=builder /sensor /sensor

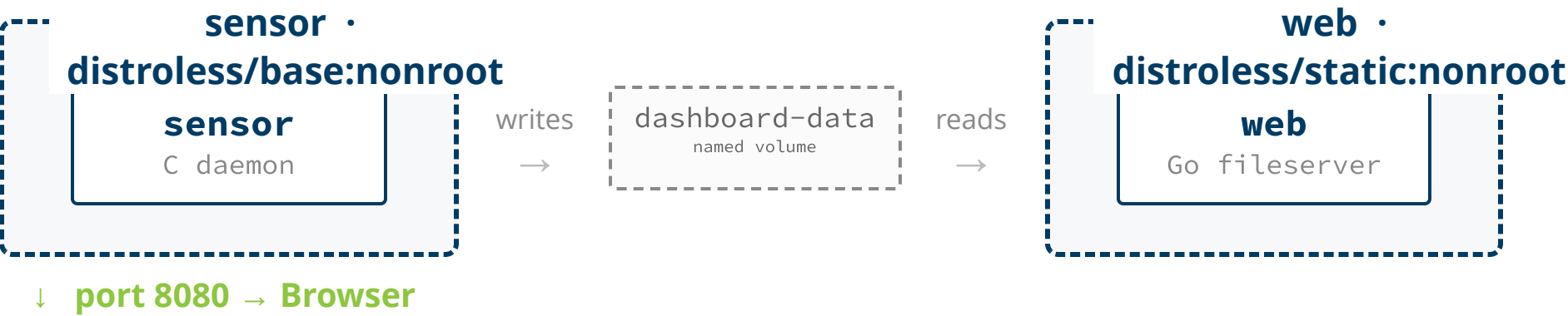
CMD ["/sensor", "/data/sensor.json"]
```

Key idea: Least privilege applies inside containers too.

STEP 4: RUNNING COMPARISON



- Same architecture as step 3 — only the image tags changed



Step	Change	Image Size
step0	Baseline: single-stage, debian:trixie, build tools included	498 MB
step1	Multi-stage: debian:trixie-slim runtime, no build tools	112 MB
step2	Microservices: sensor + nginx, named volume	101 + 112 MB = 213 MB naïve; ~112 MB on disk (shared base)
step3	Distroless: sensor (35 MB) + Go web (7.6 MB)	~40 MB on disk
step4	Nonroot: :nonroot distroless tags, uid 65532	~40 MB on disk (same size)

MULTI-ARCH

- One tag, multiple platform blobs (OCI manifest list)
 - `docker pull` selects the right arch
 - No per-device Dockerfile changes
- How:
 - `docker buildx` with a multi-platform builder
 - QEMU binfmt on build host (or native builders)
 - A registry
- Targets:
 - `linux/amd64` — dev machines
 - `linux/arm/v7` — ARM 32-bit boards (Apalis i.MX6)
 - `linux/arm64` — AArch64 boards (Verdin iMX8M Mini)

```
Makefile

REGISTRY := ghcr.io/example/dashboard
TAG      := step5
PLATFORMS := linux/amd64,linux/arm/v7,linux/arm64

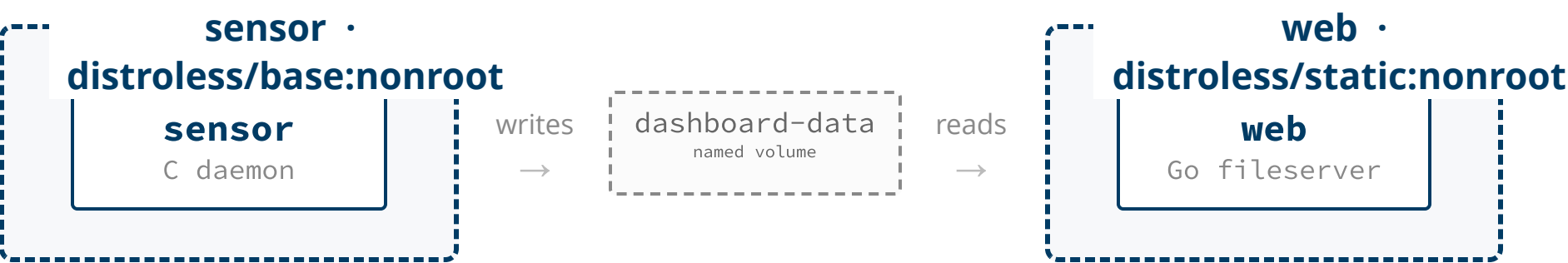
build:
    docker buildx build \
        --platform $(PLATFORMS) \
        --push \
        -t $(REGISTRY)/sensor:$(TAG) \
        sensor/
    docker buildx build \
        --platform $(PLATFORMS) \
        --push \
        -t $(REGISTRY)/web:$(TAG) \
        web/

deploy:
    scp docker-compose.yml \
        $(BOARD_USER)@$(BOARD_HOST):/tmp/
    ssh $(BOARD_USER)@$(BOARD_HOST) \
        "docker compose -f /tmp/docker-compose.yml \
        pull && up -d"
```

Key idea: One image name serves every architecture — the registry does the dispatch.

STEP 5: RUNNING COMPARISON

- Same app, same Dockerfiles — only the build command changed



↓ port 8080 → Browser · same image on amd64 / arm/v7 / arm64

Step	Change	Image Size
step0	Baseline: single-stage, debian:trixie, build tools included	498 MB
step1	Multi-stage: debian:trixie-slim runtime, no build tools	112 MB
step2	Microservices: sensor + nginx, named volume	101 + 112 MB = 213 MB naïve; ~112 MB on disk (shared base)
step3	Distroless: sensor (35 MB) + Go web (7.6 MB)	~40 MB on disk
step4	Nonroot: :nonroot distroless tags, uid 65532	~40 MB on disk (same size)
step5	Multi-arch: amd64 + arm/v7 + arm64 manifest list	~40 MB on device (right blob pulled automatically)

DEPLOYMENT & UPDATES

LAYER REUSE



- Stable base image + app layer on top
 - Only the app layer ships on update
- Anti-pattern: `RUN apt update && apt upgrade` in every build
 - Busts the cache above it
- Change one line in `sensor.c` → rebuild → push
 - Device pulls only the app layer (a few kB) of a 35 MB image

```
board terminal

$ docker pull \
  ghcr.io/example/dashboard/sensor:v2

v2: Pulling from example/dashboard/sensor
a8ca11554fce: Already exists
3d8f1b4c0b9e: Already exists
b2e7f9d1a3c4: Already exists
9f4e2b8d6a1c: Pull complete
Digest: sha256:3f7a2d...
Status: Downloaded newer image

~XX kB pulled of 35 MB total ← actual size TBD
```

Key idea: The registry is a content-addressed patch system — you only pay for what changed.

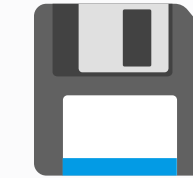
Key idea #2: Frequently-changing instructions go *last* — everything above is reused.

ONLINE & OFFLINE UPDATES



Online

- Signed, multi-arch registry
- Pull-through cache saves bandwidth
- Staggered rollout · canary devices for large fleets



Offline / Air-Gapped

- `docker save / load` via USB or SD card
- Verify signature before load
- Keep previous · roll back on health check failure

FOR BOTH

 **Health checks** — verify new container before retiring old

↔ **Rollback** — keep previous image; restart on failure

 **Signature verification** — confirm origin before load

EMBEDDED INTEGRATION LADDER



Pick the right rung for your application:

Groups & permissions

`dialout`, `gpio`, `i2c` — no kernel privileges needed

↓ *need a specific device node?*

`-device`

Single device node; nothing else exposed (`/dev/ttyUSB0`, CAN, I²C/SPI)

↓ *need a whole class of devices?*

`-device-cgroup-rule`

Device class by major number — Wayland kiosk: `c 226:* rmw, /dev/dri, -shm-size`

↓ *need kernel-level privileges?*

Linux capabilities

`-cap-drop ALL -cap-add CAP_NET_ADMIN` — surgical and auditable

↓ *still not enough?*

⊖ `-privileged`

Full host kernel access — almost never the right answer

Key idea: Expose exactly what the app needs — nothing more.

GPU INTEGRATION: NVIDIA CONTAINER TOOLKIT Torizon™

- GPU access is device passthrough at scale — not one `/dev` node, a whole driver + userspace lib stack
- `nvidia-container-toolkit` mounts host driver libs + device nodes into the container (`--runtime nvidia`)
 - Container reuses the *host's* shared objects, so driver/firmware always match
- Torizon skips that runtime: same device-cgroup mounting done directly, fully isolated — same pattern used on NXP platforms

Key idea: GPU containers aren't exotic — same device-cgroup pattern as any other embedded GPU platform, Nvidia's toolkit just automates the path list.

```
board terminal

# Jetson (L4T): nvidia-container-toolkit
$ docker run --rm --runtime nvidia \
  dustynv/l4t-pytorch:r36.2.0 \
  python3 -c "import torch; print(torch.cuda.is_available())"
```


FUTURE TOPICS



Each deserves its own talk:

Performance & Footprint

- Image size → OTA bandwidth & flash
- Startup time; avoid init overhead
- overlayfs on eMMC/SD I/O
- cgroups: pin cores, cap memory
- Pre-pull during maintenance windows

Security & Supply Chain

- Minimal images shrink CVE surface
- SBOMs (SPDX / CycloneDX)
- Image signing with Cosign / Sigstore
- Runtime hardening: read-only rootfs, drop caps
- Rootless runtimes

Debugging in Production

- Debug sidecar: share `--pid`, `--net` namespaces
- `:debug` tag adds a busybox shell
- Tooling containers: pull on demand
- Ephemeral debug containers; no restart needed

Watch this space: this 2-part series is probably becoming a 3- or 4-parter.

PUTTING IT ALL TOGETHER



⚠️ We started with...

- Single, naive image
- 498 MB · x86 only
- Runs as root
- No update strategy
- Full device access or nothing

✅ We end with...

 **Image Design** 498 MB → ~40 MB · multi-stage · 3 platforms · nonroot distroless

 **Deployment** Layer-aware OTA · online & offline · rollback on failed health check

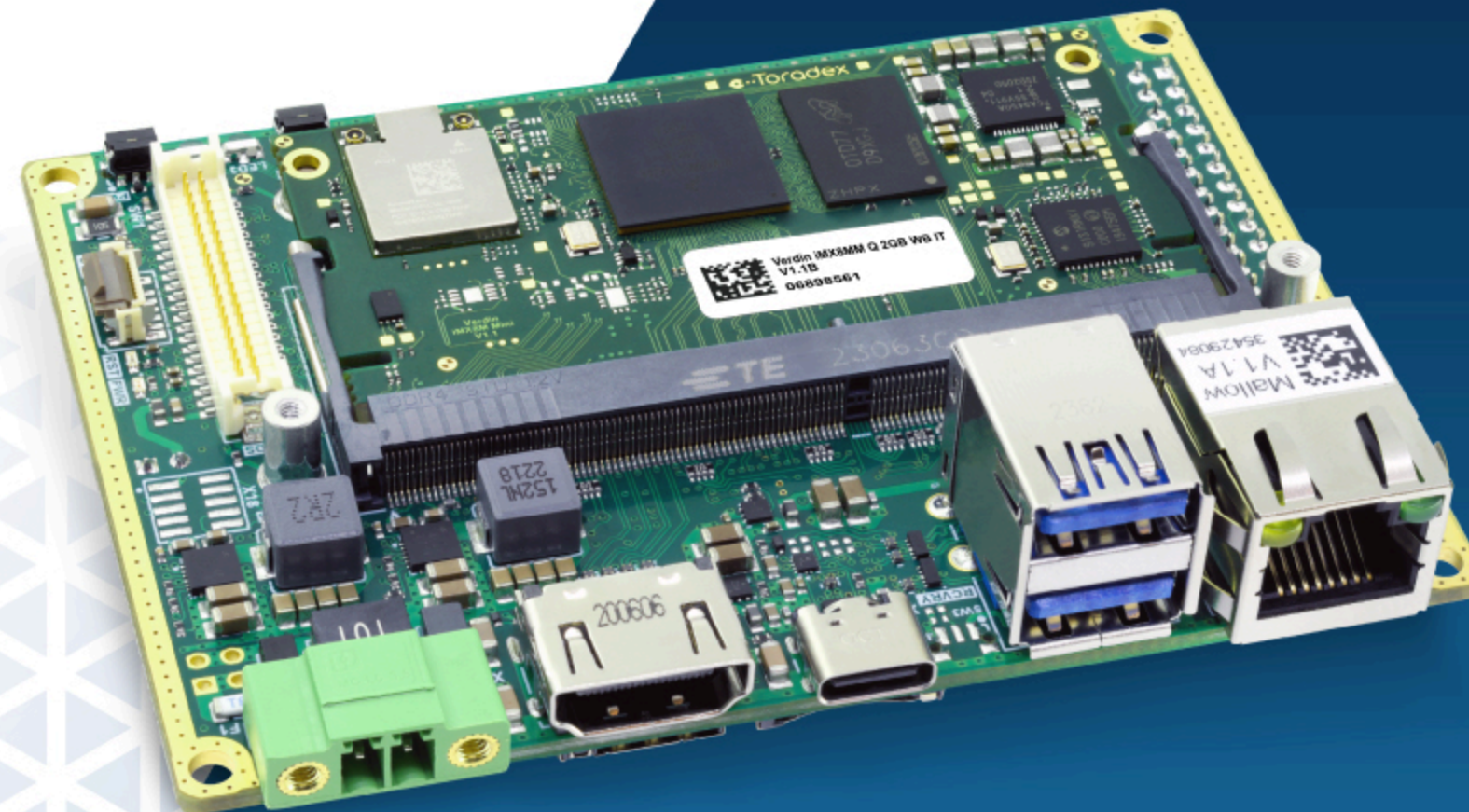
 **Integration** Least-privilege device access · capabilities over -privileged

 **Coming next** Debug sidecars · SBOMs · image signing · performance & footprint

Key takeaway: Containers *can* be right for embedded — but only if we design with embedded constraints in mind.

THANK YOU FOR YOUR INTEREST

www.toradex.com
www.torizon.io
developer.toradex.com
community.toradex.com



Torizon™
Industrial IoT. Secure. Reliable.



BACKUP SLIDES

INSPECTING LAYERS BEFORE YOU PULL



- **skopeo** reads the OCI manifest from a registry — no pull, no daemon required
- Shows layer digests + compressed sizes before committing any bandwidth
- Useful for pre-flight estimates, diffing releases, scripted fleet rollouts

```
terminal
$ skopeo inspect --raw \
  docker://ghcr.io/example/dashboard/sensor:v2 \
  | jq '.layers[] | {digest: .digest[7:19],
                    size_kb: (.size/1024|floor)}'

{ "digest": "a8ca11554fce", "size_kb": 17999 }
{ "digest": "3d8f1b4c0b9e", "size_kb": 14453 }
{ "digest": "b2e7f9d1a3c4", "size_kb": 4      }
{ "digest": "9f4e2b8d6a1c", "size_kb": XX    } ← TBD
```

Key idea: Know your update payload size *before* you commit the device's bandwidth.

THE CONTAINER ECOSYSTEM



OCI — The Common Foundation

- **Image Format Spec** — content-addressed layers; portable across all tools
- **Runtime Spec** — defines how containers execute on Linux
- **Distribution Spec** — how registries serve and receive images

Runtimes

- **containerd** — lightweight daemon; the engine inside Docker and Torizon
- **Docker Engine** — containerd + CLI + UX layer; ideal for development
- **Podman** — daemonless, rootless-native; compatible CLI

Build

- **docker buildx** — multi-platform builds
- **buildah** — daemonless; scriptable

Registry & Inspection

- **docker** — pull, push, inspect
- **skopeo** — registry ops without a daemon
- **crane** — scripting-friendly registry client

Security

- **cosign** — sign & verify images (Sigstore)
- **syft / gype** — SBOM generation & CVE scanning

We use **Docker** throughout — but the images and workflows are portable across the whole stack.